实验二 时序逻辑设计

一、实验目的

1 掌握 Verilog 语言和 Vivado、Logisim 开发平台的使用；

2 掌握基础时序逻辑电路的设计和测试方法。

二、实验内容（使用 Logisim 或 Vivado 实现）

1 锁存器、触发器的设计与测试

2 寄存器、计数器的设计与测试

3 状态机的设计与测试

三、实验要求

1 掌握 Vivado 或 Logisim 开发工具的使用，掌握以上电路的设计和测试方法；

2 记录设计和调试过程（Verilog 代码/电路图/表达式/真值表，Vivado 仿真结果，Logisim 验证结果等）；

3 分析 Vivado 仿真波形/Logism 验证结果，注重输入输出之间的对应关系。

四、实验过程及分析

（一）S-R 锁存器：

1、代码：

```
module S_R_latch(Q, QN, S, R, EN, RST);
    input S, R;
    input EN, RST;
    output reg Q, QN;
    always @(S, R, EN, RST) begin
        if (RST) begin
            Q = 0; QN = 1;
        end
        else begin
            case({S, R})
            2'b00: begin
                // S=0, R=0：保持状态不变
                Q <= Q;
                QN <= QN;
            end
            2'b01: begin
                // S=0, R=1：复位状态，Q=0, QN=1
                Q <= 0;
                QN <= 1;
```

```
            end
            2'b10: begin
                // S=1, R=0：置位状态，Q=1, QN=0
                Q <= 1;
                QN <= 0;
            end
            2'b11: begin
                // S=1, R=1：非法状态，两者都置 0
                Q <= 0;
                QN <= 0;
            end
            endcase
        end
    end
endmodule
```

分析：该模块实现一个带异步复位的 S-R 锁存器，具有两个输入 S、R 和使能信号 EN 以及复位信号 RST。输出 Q 和 QN 互为反相。always @(S, R, EN, RST) 表明当 S、R、EN 或 RST 信号变化时更新状态。RST 的优先级最高，当复位信号 RST 为高时，将 Q=0、QN=1，确保锁存器被清零。否则，根据 {S,R} 的组合决定输出：

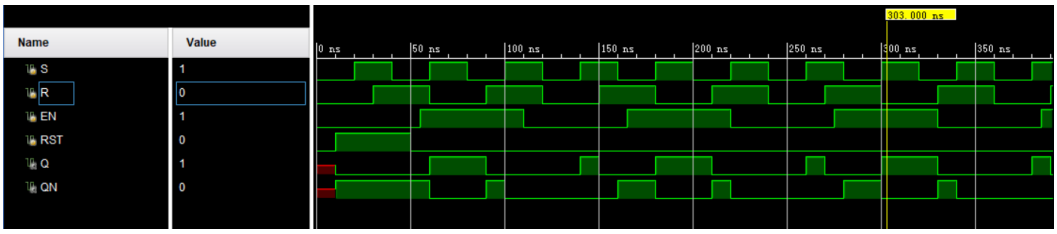
- S=0, R=0（2'b00）： 保持当前状态（Q<=Q, QN<=QN）。
- S=0, R=1（2'b01）： 复位状态，令 Q<=0, QN<=1。
- S=1, R=0（2'b10）： 置位状态，令 Q<=1, QN<=0。
- S=1, R=1（2'b11）： 非法状态，将 Q,QN 都置 0（无效输出）。

2、测试文件：

```
module sim_S_R_latch;
    reg S, R, EN, RST;
    wire Q, QN;
    S_R_latch SR (.Q(Q), .QN(QN), .S(S), .R(R), .EN(EN), .RST(RST));
    initial fork
        S = 0; R = 0; EN = 0; RST = 0;
        #10 RST = 1;
        #50 RST = 0;
        forever #20 S = ~S;
        forever #30 R = ~R;
        forever #55 EN = ~EN;
    join
endmodule
```

测试策略： 初始时令 RST=1，对锁存器进行复位，观察输出是否快速变为 Q=0、QN=1；然后将 RST 拉低，以解除复位。将 S 和 R 分别以不同周期翻转，确保覆盖到 {S,R} 的所有组合。特别关注以下情形：S=1,R=0（应置位）；S=0,R=1（应复位）；S=0,R=0（输出应保持不变）；S=1,R=1（非法，输出为 0）。

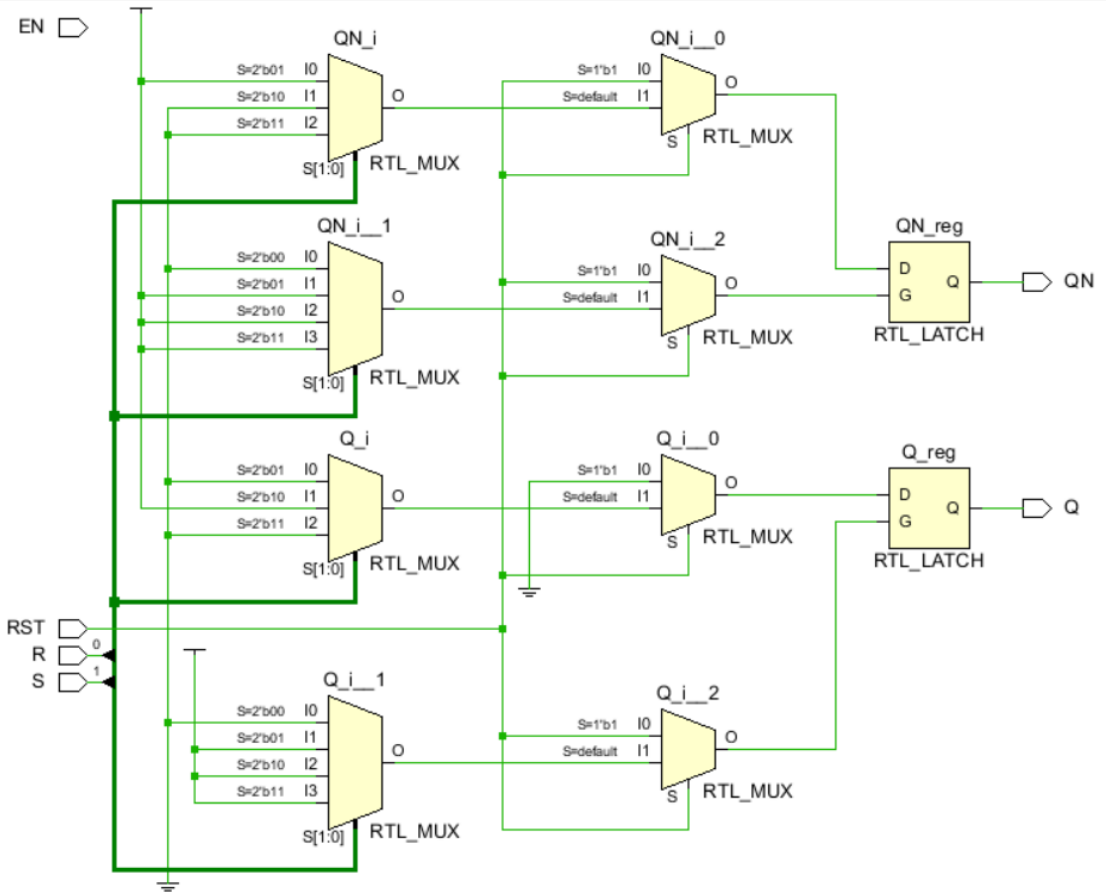
3、SIMULATION：



分析：观察仿真波形可以发现：在第一个复位周期（RST 拉高）结束时，Q 被置为 0，QN 为 1，符合复位状态预期。复位信号结束后，S 和 R 按设定的周期开始变化：

- 当 S=1, R=0 时，输出 Q 被置为 1，QN 变为 0（置位操作）。
- 当 S=0, R=1 时，输出 Q 被置为 0，QN 变为 1（复位操作）。
- 当 S=0, R=0 时，Q 和 QN 保持原来的值（无变化）；
- 当 S=1, R=1 时，电路进入非法状态，此时波形中可见 Q=0，QN=0。

4、RTL ANALYSIS：



（二）D 锁存器：

1、代码：

```
module D_latch(Q, QN, D, EN, RST);
    output reg Q, QN;
    input D;
    input EN, RST;

    always @(EN, RST, D) begin
        if(RST) begin
            Q = 0;
            QN = 1;
        end
        else if(EN) begin
            Q <= D;
            QN <= ~D;
        end
    end
endmodule
```

分析：该模块实现一个带使能和异步复位的 D 锁存器。输入信号 D 为数据输入，EN 为使能，RST 为复位，Q 为输出，QN 为反相输出。always @(EN, RST, D) 表示当 EN、RST 或 D 信号变化时更新。首先，如果 RST 为高，则清零输出（Q=0, QN=1）。否则，如果 EN 为高，则将 Q 置为 D，QN 置为 ~D；若 EN 为低，则由于没有其他赋值语句，输出保持不变，实现锁存功能。使用非阻塞赋值 <= 来更新 Q，确保时序一致性。

2、仿真代码：

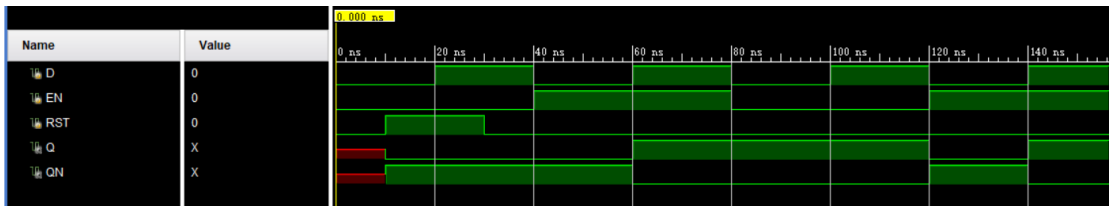
```
module sim_D_latch;
    reg D, EN, RST;
    wire Q, QN;
    D_latch d (.Q(Q), .QN(QN), .D(D), .EN(EN), .RST(RST));

    initial fork
        D = 0; EN = 0; RST = 0;
        #10 RST = 1;
        #30 RST = 0;
        forever #20 D = ~D;
        forever #40 EN = ~EN;
        #10 RST = 1;
        #50 RST = 0;
```

```
    join
endmodule
```

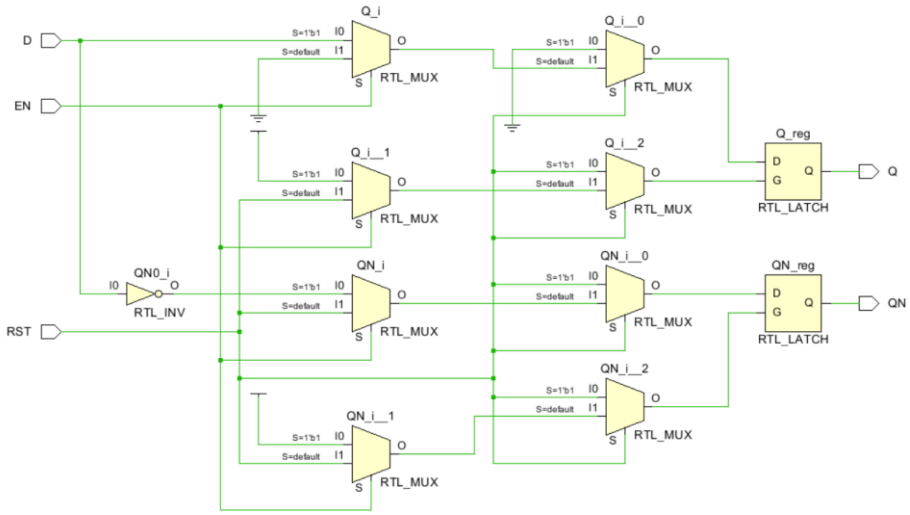
仿真策略： 初始时将 RST 拉高，观察 Q 是否被置为 0；之后将 RST 拉低解除复位。设置 EN=1，当时钟或触发条件允许时，D 的变化应立即反映到 Q 上；然后将 EN=0， D 的变化不应影响 Q，设置时钟周期反复翻转 EN。在 EN=1 时令 D 以一定周期翻转，确保 Q 能及时更新到 D 的当前值；在 EN=0 时继续翻转 D，确认 Q 不变。

3、SIMULATION：



分析：在初始复位周期结束时，Q=0，QN=1。复位解除后，当 EN=0 时，即使 D 信号翻转，Q 保持不变；当 EN 从 0 变为 1 时，随后的 D 翻转在下一个事件点被捕获，Q 也随之更新。例如，可以看到在 EN 拉高期间，Q 与 D 同步切换；而在 EN=0 时段，无论 D 如何变化，Q 都保持上一次的值。所有仿真波形与预期一致，验证了 D 锁存器的功能。

4、RTL ANALYSIS：



（三）D 触发器：

1、代码：

```
module D_ff(Q, QN, D, EN, RST, CLK);
    output reg Q, QN;
    input D;
    input EN, RST, CLK;

    always @(posedge CLK) begin //同步复位
    // always @(posedge CLK, posedge RST) begin  异步复位
```

```
        if (RST) begin
            Q <= 1'b0;
            QN <= 1'b1;
        end
        else if (EN) begin
            Q <= D; QN <= ~D;
        end
    end
endmodule
```

分析：该模块为一个带使能和同步复位的 D 触发器。输入 D 为数据信号，EN 为使能，RST 为同步复位，CLK 为时钟。always @(posedge CLK) 表示仅在时钟上升沿更新；在时钟沿时，若 RST=1，则将 Q 置为 0，QN 置为 1（同步复位）；否则若 EN=1，则将 Q 赋值为 D，QN=~D；若 EN=0，则因为没有赋值操作，Q 保持不变。与 D 锁存器不同，D 触发器只有在上升沿才更新，因此 Q 只有在特定时刻才发生变化。

同步复位与异步复位的区别：同步复位指复位信号必须在时钟上升沿到来时才起作用，电路只有在时钟触发的那一刻才会被清零；而异步复位则不依赖时钟信号，只要复位信号有效，电路立即复位。同步复位的时序一致性好、综合方便，但响应速度慢；异步复位响应快，可在任何时刻清零，但可能引入亚稳态或毛刺问题。

如何实现使能+复位，哪个优先级更高：

```
        if (RST) begin
            Q <= 1'b0;
            QN <= 1'b1;
        end
        else if (EN) begin
            Q <= D; QN <= ~D;
        end
```

这段代码实现了使能+复位，其中复位优先级更高，若出现复位信号，无论使能如何都将触发器复位。

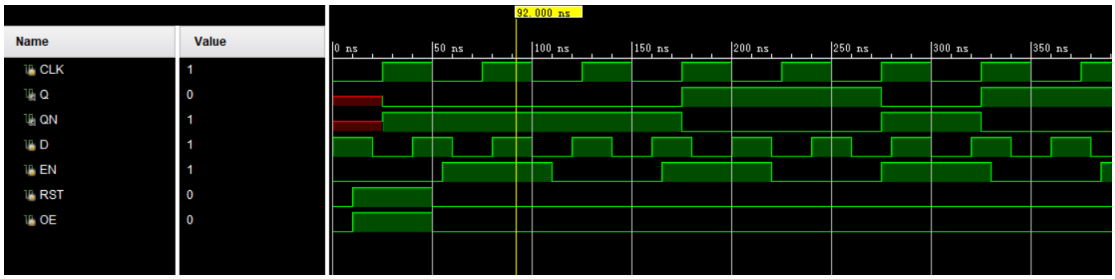
2、仿真代码：

```
module sim_D_ff;
    reg CLK;
    wire Q, QN;
    reg D, EN, RST, OE;
    D_ff D_ff_1(Q, QN, D, EN, RST, CLK);
    initial fork
        D = 1; EN = 0; RST = 0; CLK = 0; OE = 0;
        #10 begin RST = 1; OE = 1; end
```

```
        #50 begin RST = 0; OE = 0; end
        forever #55 EN = ~EN;
        forever #20 D = ~D;
        forever #25 CLK = ~CLK;
    join
endmodule
```

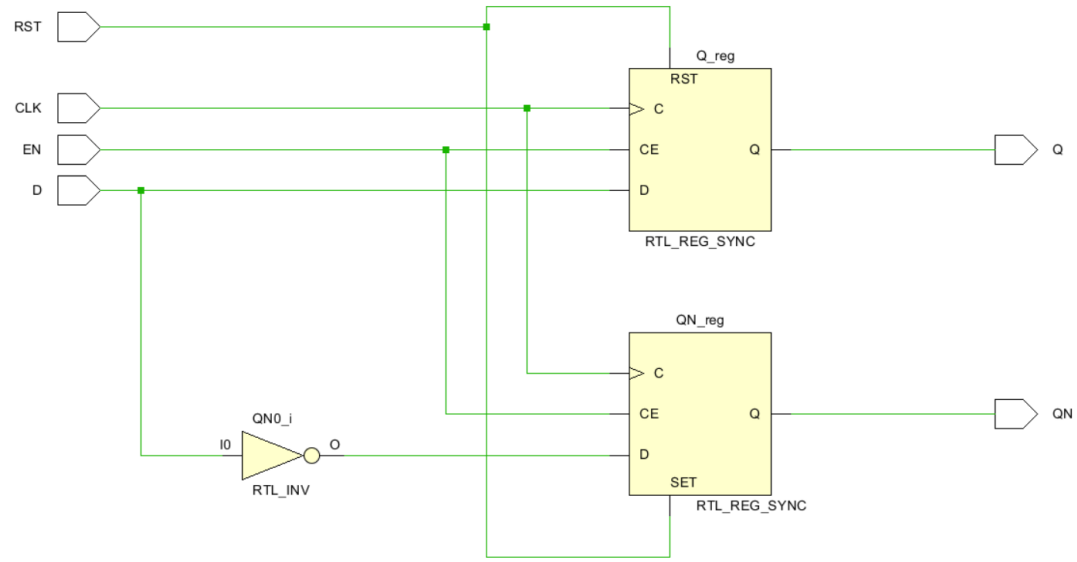
仿真策略：将 RST 置高，在一个上升沿到来时观察 Q 被清零；之后将 RST 置低。在 RST 解除后，通过周期性翻转 EN 和输入 D，验证如下情况：在 EN=1 时，每个时钟上升沿 Q 应更新为 D；在 EN=0 时，尽管 D 可能变化，但 Q 不应改变。生成稳定的时钟信号不断翻转，并在多个周期内观察 Q 的行为，检查触发器的边沿敏感特性，在仿真中使用多个时钟周期充分测试 EN 的切换和复位的优先级。

3、SIMULATION：



分析：在第一个时钟上升沿时（RST=1），Q 被置为 0（同步复位）；复位解除后，随着时钟连续变化，我们看到在 EN=1 时每次上升沿 Q 跟随输入 D 变化；在 EN=0 时段，即使 D 在时钟边沿间发生了翻转，输出 Q 在下一个时钟沿仍保持之前的值，没有更新。这验证了 D 触发器仅在满足使能且有时钟上升沿时才更新的特性。输出波形与设计预期完全一致。

4、RTL ANALYSIS：



（四）寄存器：

1、代码：

```
module register(Q, D, OE, CLK);
    parameter N = 8;
    output reg [N-1:0] Q;
    input [N:1] D;
    input OE, CLK;

    always @(posedge CLK or posedge OE)
        if (OE) Q <= 8'bzzzz_zzzz;
        else Q <= D;
endmodule
```

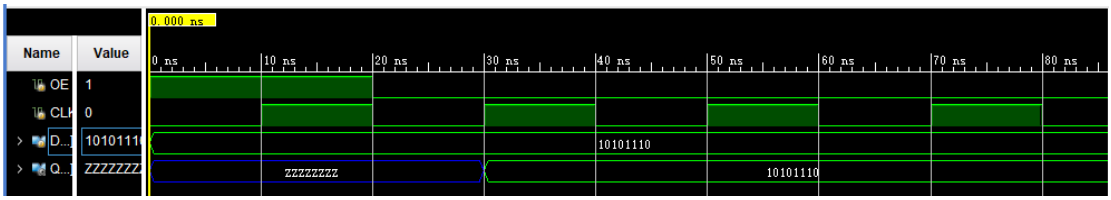
分析：该模块为一个 8 位带输出使能的寄存器。输入 D 为 8 位数据，CLK 为时钟，OE 为输出使能。always @(posedge CLK or posedge OE) 表示该模块在时钟上升沿或 OE 的上升沿时触发。如果 OE=1，则将输出 Q 置为高阻态（8'bzzzz_zzzz）；否则，在时钟上升沿将 D 的值装入 Q。因此，该寄存器在 OE 信号高时输出被禁用，在 OE 低时随时钟更新数据。

2、仿真文件：

```
module sim_register;
    reg OE, CLK;
    reg [7:0] D;
    wire [7:0] Q;
    register test(.OE(OE), .CLK(CLK), .D(D), .Q(Q));
    initial fork
        OE = 1;
        CLK = 0;
        D = 8'b1010_1110;
        #20 OE = ~OE;
    join
    always #10 CLK = ~CLK;
endmodule
```

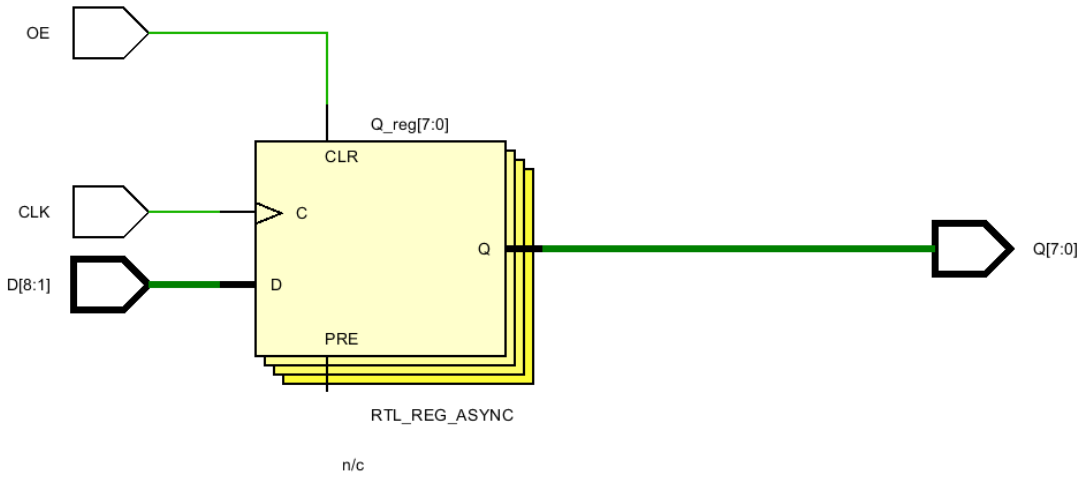
测试策略：初始令 OE=1，观察 Q 是否立即变为高阻态。将 OE 拉低后，在下一次时钟上升沿前提供稳定的 D 数据，在时钟上升沿之后观察 Q 是否正确更新为 D 的值。在不同的时钟周期重复上述操作，验证寄存器对 OE 信号的响应和对时钟的同步写入功能。

3、SIMULATION：



分析：在 OE = 1 时，Q 被赋为高阻态；随后将 OE 拉低，在下一个上升沿 Q 更新为输入 D 的值 1010_1110，并保持不变。整个过程中，寄存器的行为与设计一致：OE 为 1 时输出禁用，为 0 时在时钟边沿捕获数据。高阻态在波形中清晰可见，说明寄存器使能功能正常。

4、RTL ANALYSIS：



（五）移位寄存器：

1、代码：

```
module shift_register(S1, S0, D, Dsl, Dsr, Q, CLK, CR);
    parameter N = 4;
    input S1, S0;
    input Dsl, Dsr;
    input CLK, CR;
    input [N-1:0] D;
    output reg [N-1:0] Q;

    always @(posedge CLK or posedge CR) begin
        if(CR)
            Q <= 0;
        else
            case({S1, S0})
                2'b00: Q <= Q;
                2'b01: Q <= {Dsr, Q[N-1:1]};
                2'b10: Q <= {Q[N-2:0], Dsl};
```

```
            2'b11: Q <= D;
        endcase
    end
endmodule
```

分析：该模块实现一个 4 位可配置的移位寄存器，具有并行载入和左右移位功能。输入 S1、S0 作为操作控制信号，输入 D 为 4 位并行数据，Dsl 为左移时补入的最低位数据，Dsr 为右移时补入的最高位数据，CLK 为时钟，CR 为异步清零。电路中：若 CR=1（高电平），在时钟上升沿立即将 Q 清零；否则根据 S1S0 的组合决定移位操作：

- {S1, S0} = 2'b00：保持状态（Q<=Q）。
- {S1, S0} = 2'b01：右移操作（Q<= {Dsr, Q[N-1:1]}），即最高位由 Dsr 填充，其余位右移一位。
- {S1, S0} = 2'b10：左移操作（Q<= {Q[N-2:0], Dsl}），即最低位由 Dsl 填充，其余位左移一位。
- {S1, S0} = 2'b11：并行载入（Q<=D），在一个时钟沿将外部输入 D 全部装入 Q。

2、仿真文件：

```
module sim_shift_register;
    reg S1, S0, Dsl, Dsr, CLK, CR;
    reg [3:0] D;
    wire [3:0] Q;

    shift_register sft
(.S1(S1), .S0(S0), .D(D), .Dsl(Dsl), .Dsr(Dsr), .Q(Q), .CLK(CLK), .CR(CR));

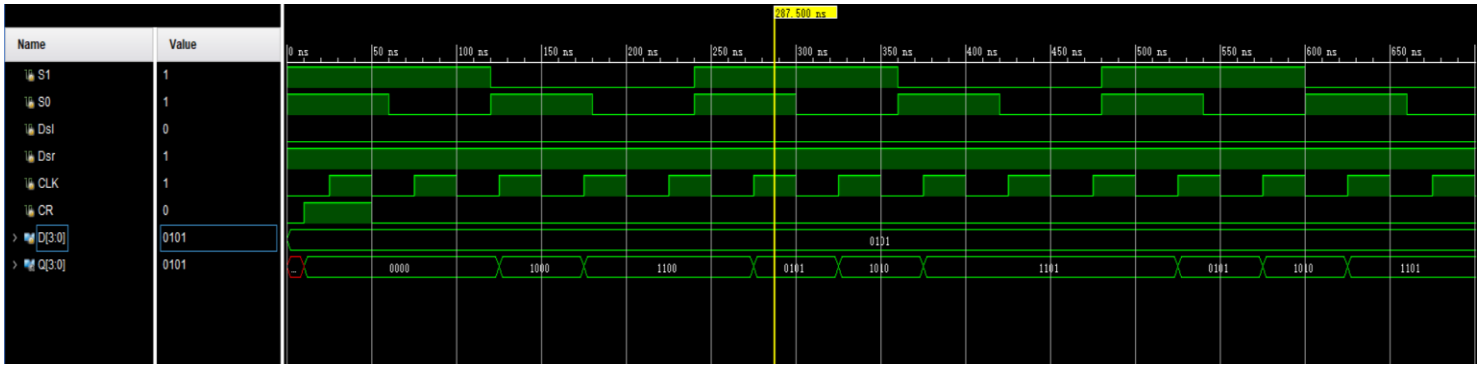
    initial fork
        S1 = 1; S0 = 1; Dsl = 0; Dsr = 1; CLK = 0; CR = 0; D = 4'b0101;

        #10 CR = 1;
        #50 CR = 0;

        forever #25 CLK = ~CLK;
        forever #60 S0 = ~S0;
        forever #120 S1 = ~S1;
    join
endmodule
```

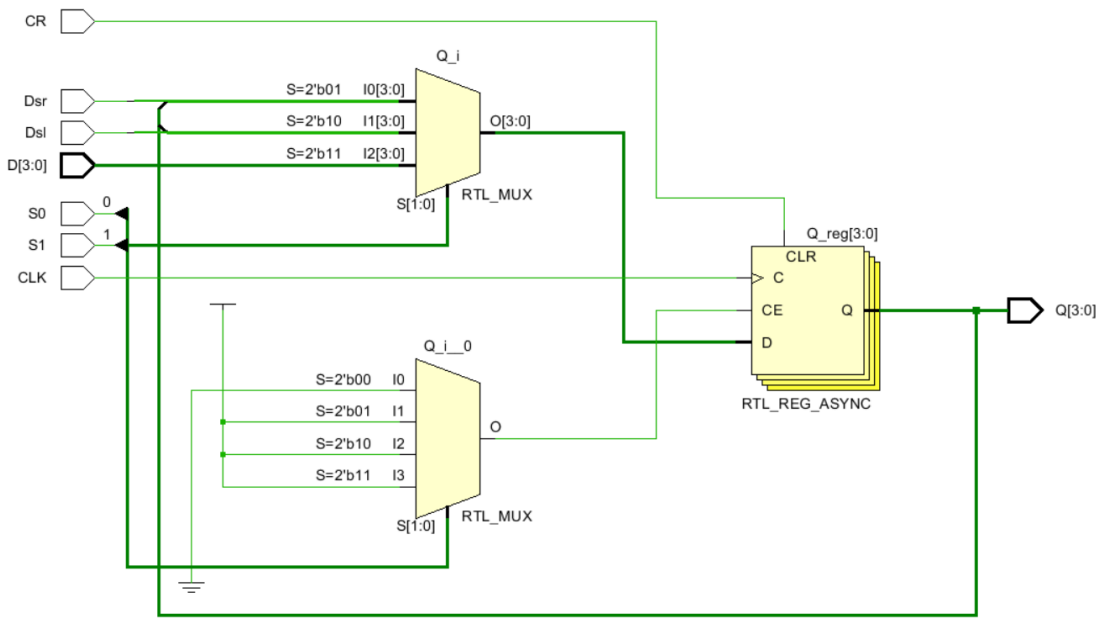
仿真策略：初始时将 CR 拉高清零，观察 Q 被置零，随后将 CR 置低电平。初始时设定 S1S0=11，按照一定周期交替翻转两位，观察四种不同状态的表现。右移时 S1S0=01，在每个时钟周期 Q 右移。通过给定 Dsr 观察最高位依次被 1 填充，验证右移行为。左移时 S1S0=10，在每个时钟周期 Q 左移。通过给定 Dsl 观察最低位依次被 0 填充，验证左移行为。加载时 S1S0=11，通过将 D 设定为某一常数，在一个时钟沿将其并行加载到 Q 中，观察 Q 的值验证加载是否正确。保持时 S1S0=00，在时钟来临时 Q 保持原值，无变化。在仿真中依次切换 S1S0 信号，通过时钟脉冲观察 Q 的变化：先测试清零，然后依次切换到并行加载、左移、右移、保持等模式，确保每种操作都得到覆盖。

3、SIMULATION:



分析：在第一个时钟边沿时，由于 CR=1，Q 被清零。之后，当 S1S0 切换到 01（右移模式）时，每个时钟周期 Q 的高位由 Dsr 填充，位模式右移：Q 初始为 0000，右移后变为 1000，再右移为 1100 。切换到 11（并行加载）时，在一个时钟沿 Q 立即变为输入 D 的值 0101，验证了并行载入功能。切换到 10（左移模式）后，Q 的低位由 Dsl 填充，高位左移：如 Q 从 0101 到 1010。保持模式（00）下，无论时钟如何变化，Q 均保持原值。可知，移位寄存器的仿真行为符合设计要求，各模式功能正常。

4、RTL ANALYSIS:



（六）68 进制计数器：

1、代码：

```
module counter74x161(CEP, CET, PE, CLK, CR, D, TC, Q);
    parameter N = 8;
    parameter M = 68;
    input CEP, CET, PE, CLK, CR;
    input [N-1:0] D;
    output reg TC;
    output reg [N-1:0] Q;
```

```
    wire CE;
    assign CE = CEP & CET;
    always @(posedge CLK, negedge CR)
    if(~CR) begin
        Q <= 0;
        TC <= 0;
    end
    else if(~PE)
        Q <= D;
    else if(CE) begin
        if(Q == M-1) begin
            TC <= 1;
            Q <= 0;
        end
        else begin
            Q <= Q + 1;
            TC <= 0;
        end
    end
    else
        Q <= Q;
endmodule
```

分析：该模块为一个带并行装载的模 68 计数器。输入信号包括级联使能 CEP、CET，预置使能 PE（低有效并行装载），时钟 CLK，清零 CR（低有效），并行数据 D。输出为 8 位计数器 Q 和进位输出 TC。内部信号 CE = CEP & CET 为级联使能。在每个时钟上升沿或 CR 的下降沿触发：

- 如果 CR=0，立即清零：Q<=0，TC<=0。
- 否则若 PE=0，则将输入 D 并行装入 Q，TC 保持不变。
- 否则若 CE 为真，则进行计数：如果 Q == M-1，则将 TC 置 1，Q 归零；否则 Q<=Q+1，TC<=0。
- 如果 CE=0 且 PE=1，则保持当前计数 Q 不变。

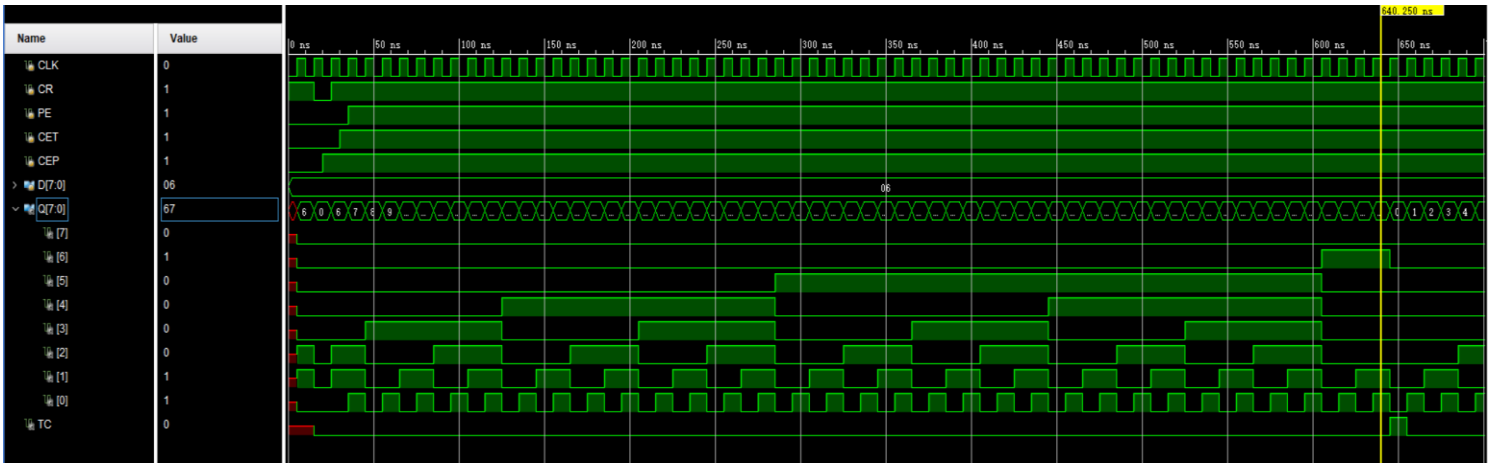
2、仿真文件：

```
module sim_counter74x161;
    reg  CLK, CR, PE, CET, CEP;
    reg  [7:0] D;
    wire [7:0] Q;
    wire TC;
    counter74x161 count (CEP, CET, PE, CLK, CR, D, TC, Q);
    initial fork
        CLK = 0; CR  = 1; PE  = 1; CET = 0; CEP = 0; D = 8'h06;
        forever #5 CLK = ~CLK;
        #15 CR = 0;
        #25 CR = 1; PE = 0;
```

```
        #35 PE = 1;
        #30 CET = 1;
        #20 CEP = 1;
    join
endmodule
```

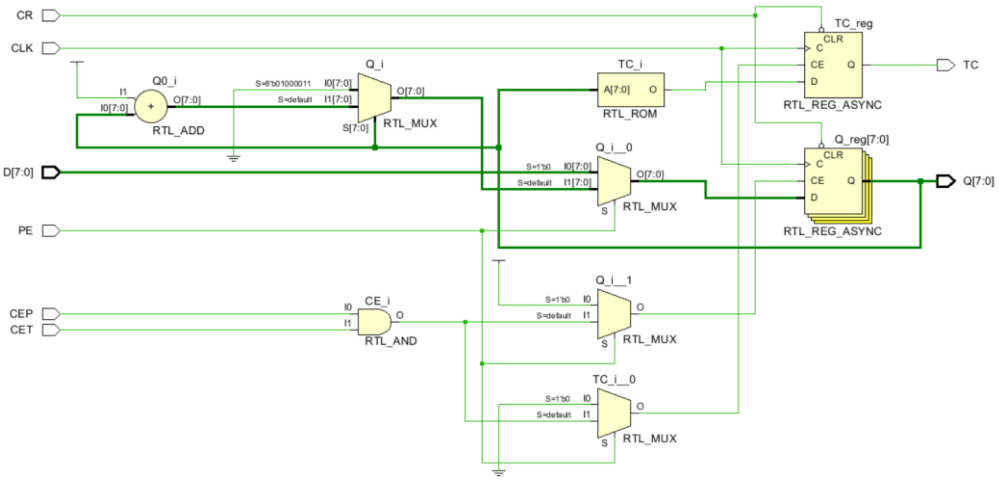
测试策略： 在仿真开始时给予一个清零脉冲，观察 Q 立即变为 0，TC 为 0。在 CR 解除后，拉低 PE，设定一个并行输入 D=8'h06，在下一个时钟上升沿观察 Q 是否变为 6。保持 PE=1，逐步使能 CEP 和 CET，使 CE 变为 1，此时 Q 开始计数。在每个时钟上升沿，Q 从并行装入的值开始递增，TC 保持 0，直至 Q 达到 67。当 Q 达到 67 时，观察在下一个时钟上升沿是否将 TC 置 1 并使 Q 回零；继续计数时 TC 应恢复 0，Q 从 0 开始新一轮计数。

3、SIMULATION：



分析：在 t=15ns 处 CR 被拉低触发异步清零，此后 Q=0，TC=0。随后的 t=25ns 将 PE 拉低并提供 D=6，在 t=25ns 的时钟沿 Q 被装载为 6 ，此时 TC 仍为 0。此后，将 CEP 和 CET 依次拉高，当两个使能都为 1 时计数器进入计数状态，Q 在每个时钟上升沿依次自增（6->7->8…）。在仿真窗口中可见，Q 从 6 开始增加，TC 在达到 67 时被置为 1 并且 Q 回归 0。计数过程中，当 CE=0 时 Q 保持不变，符合保持功能。整个波形符合模 68 计数器的设计：并行加载、逐计数和进位输出均正常。

4、RTL ANALYSIS：



（七）3 频计数器：

1、代码：

```
module FSM_case(input clk, input reset, output y);
    reg [2:0] state, nextstate;
    always @(posedge clk, posedge reset) begin
        if(reset) begin
            state <= 3'b001;
            nextstate <= 3'b010;
        end
        else state <= nextstate;
    end
    always @(posedge clk) begin
        case(state)
            3'b001: nextstate <= 3'b010;
            3'b010: nextstate <= 3'b100;
            3'b100: nextstate <= 3'b001;
            default: nextstate <= 3'b001;
        endcase
    end
    assign y = (state == 3'b001);
endmodule
```

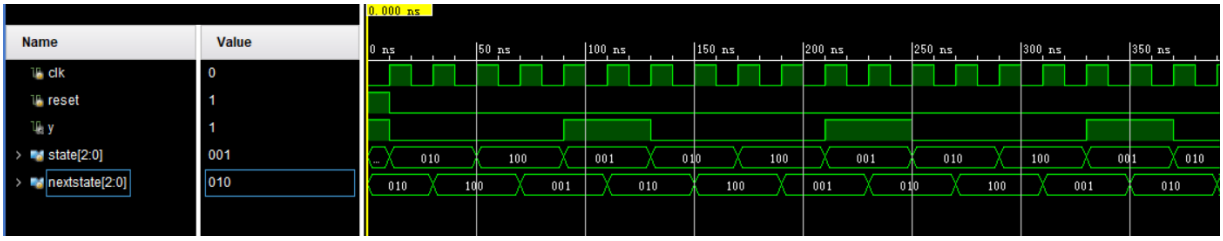
分析：该模块实现一个基于状态机的 3 分频器。使用一个 3 位寄存器 state 表示当前状态，以及 nextstate 作为下一个状态。复位时，state 被初始化为 3’b001，nextstate = 3’b010。之后在每个时钟上升沿上状态按以下循环转移：001→010→100→001……。输出信号 y = (state == 3’b001)，即当状态为 001 时为高，否则为低。因此，每经过 3 个时钟周期，输出 y 会产生一个高脉冲，实现时钟分频。

2、仿真文件：

```
module sim_FSM_case;
    reg clk, reset;
    wire y;
    FSM_case fsm(clk, reset, y);
    initial fork
         clk = 0; reset = 1;
         forever #10 clk = ~clk;
         #10 reset = 0;
    join
endmodule
```

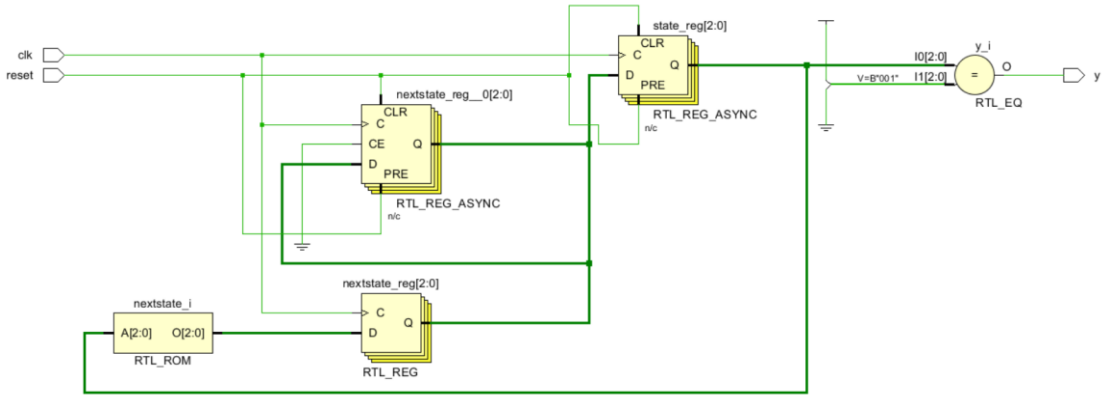
测试策略：初始阶段保持 reset=1，在一个时钟上升沿后确认 state=001、y=1；然后释放 reset。在释放复位后，继续输入稳定时钟信号，观察状态循环：001→010→100→001，输出 y 每当状态变为 001 时为高。通过观察 y 的波形，验证每三个时钟周期产生一个脉冲。

3、SIMULATION：



分析：仿真波形显示：在复位阶段，state 被置为 001，此时输出 y=1。复位结束后，状态随着时钟变化依次为 010（y=0）、100（y=0）、再回到 001（y=1），然后重复此循环。输出 y 的高电平只在 state=001 时出现，其余周期均为 0。这样每间隔两个时钟周期输出一个高脉冲，实现了 3 分频功能。波形证明计数器的状态机正常工作，输出周期与设计一致。

4、RTL ANALYSIS：



（八）二进制数 10 序列检测器：

1、代码：

```
module FSM_cf(input clk,
              input reset,
              input a,
              output y);
    reg [1:0] state, nextstate;
    always @(posedge clk, posedge reset) begin
        if(reset) begin
            state <= 2'b00;
            nextstate <= 2'b00;
        end
        else begin
            state <= nextstate;
        end
    end
    always @(*) begin
        case(state)
```

```
            2'b00: if(a) nextstate <= 2'b01;
                   else nextstate <= 2'b00;
            2'b01: if(a) nextstate <= 2'b01;
                   else nextstate <= 2'b10;
            2'b10: if(a) nextstate <= 2'b01;
                   else nextstate <= 2'b00;
            default: nextstate <= 2'b00;
        endcase
    end
    assign y = (state == 2'b10);
endmodule
```

分析：该模块实现一个检测输入序列中“10”模式的有限状态机。使用 2 位状态寄存器 state 和 nextstate，复位时 state=2’b00。状态转移逻辑如下：

- 状态 00：如果输入 a=1，则 nextstate=01，输入 a=0 则保持在 00。
- 状态 01：如果 a=0，则检测到“10”模式，设置 nextstate=10；否则保持在 01。
- 状态 10：如果 a=0，则回到 00，如果 a=1，则回到 01。

输出 y = 1 当且仅当 state == 10，否则 y=0。这样，当 FSM 从状态 01 接收到输入 0 时进入状态 10，并在此状态输出高电平，表示检测到一个“10”模式。

2、仿真文件：

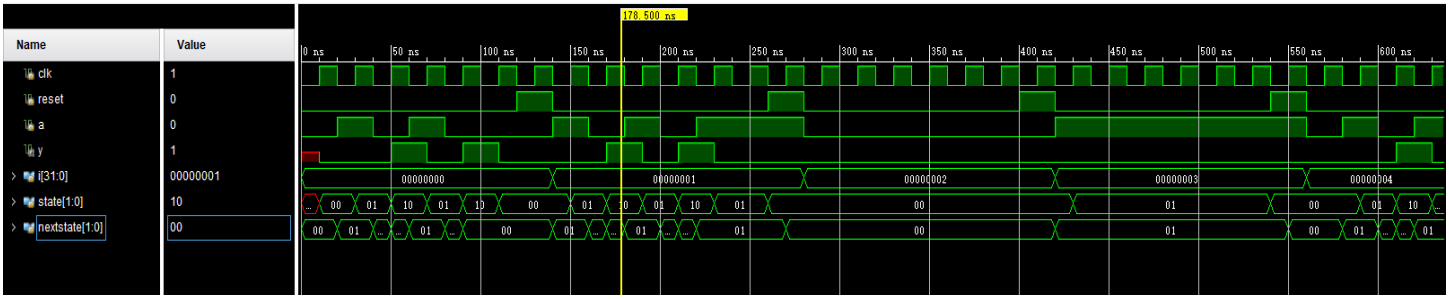
```
module sim_FSM_cf;
    reg clk, reset, a;
    wire y;
    integer i;
    FSM_cf fsm(clk, reset, a, y);
    initial fork
        clk = 0; reset = 1; a = 0;
        forever #10 clk = ~clk;
        for (i = 0; i < 5; i = i + 1) begin
            case(i)
                0: begin
                    // 测试序列: 0,1,0,1,0
                    a = 0; #20;
                    a = 1; #20;
                    a = 0; #20;
                    a = 1; #20;
                    a = 0; #20;
                end
                1: begin
                    // 测试序列: 1,0,1,0,1
                    a = 1; #20;
                    a = 0; #20;
                    a = 1; #20;
                    a = 0; #20;
```

```
                a = 1; #20;
            end
            2: begin
                // 测试序列: 连续 0
                a = 0; #100;
            end
            3: begin
                // 测试序列: 连续 1
                a = 1; #100;
            end
            4: begin
                // 测试序列: 交替变化
                repeat (10) begin
                    a = ~a; #20;
                end
            end
        endcase
        #20 reset = 1;
        #20 reset = 0;
    end
  join
endmodule
```

测试策略：使用多组输入序列测试检测器：

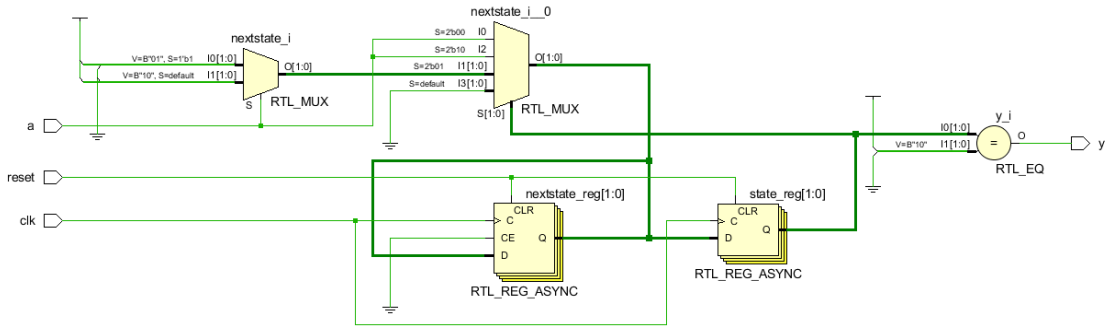
- 固定序列测试：输入序列 {0,1,0,1,0} 和 {1,0,1,0,1}，检查输出 y 在出现“10”时刻是否拉高。
- 无目标序列测试：长时间全为 0 或全为 1 序列，验证 y 始终为 0（无误检）。
- 交替序列测试：生成更长的交替 0/1 序列，观察在每次出现“10”时输出 y 的响应。
- 复位测试：每完成一个序列测试后，施加一次 reset 脉冲，验证 FSM 能够重新初始化并继续检测。

3、SIMULATION：



分析：在输入序列 {0,1,0,1,0} 中，第 3 个时钟时检测到模式“10”，此时状态从 01 跳到 10，输出 y=1；接着第 5 个时钟再次检测到“10”，y 再次为 1。对于序列 {1,0,1,0,1}，同样检测到“10”，输出 y 在相应时刻拉高。对于全 0 或全 1 的序列，状态机未检测到任何“10”，输出 y 一直保持 0。对于长时间交替序列，仿真显示每当输入出现“10”对（即先为 1 后为 0）时，输出 y 在对应时钟周期为高脉冲，其余时间为 0。所有测试表明，序列检测器准确检测输入流中的“10”模式且无误报，与预期功能一致。

4、RTL ANALYSIS:



五、调试和心得体会

1、问题与调试方法：

在实验过程中，常遇到仿真输出异常或功能不符的问题。例如编写 S-R 锁存器时忘记在 case 外处理复位信号，导致复位无效；或在寄存器模块中对 OE 的高阻处理不熟悉，导致波形中看不到输出被置高阻。遇到此类问题时，通过仔细阅读代码逻辑和时序图，并逐步使用仿真工具检查信号时序来定位错误。

调试时发现，如果没有正确使用复位或初始化，输出值可能是未知态（x），影响后续逻辑。务必在仿真激励中先施加复位信号，并在代码中对所有寄存器给出明确的复位条件。对同步复位和异步复位的使用也需分清优先级与时序差异。

2、学习收获：

本实验加强了我对时序逻辑设计的理解。通过编写 Verilog 实现不同的时序电路，深刻认识到非阻塞赋值（<=）和阻塞赋值（=）在时序电路中的区别；明白了锁存器和触发器的区别及使用场景；掌握了并行装载计数器、可编程移位寄存器、FSM 等复杂模块的设计思路。仿真过程中，通过观察波形与预期状态的对应关系，学会了从时序波形中找出设计逻辑的问题。

3、总结：

本次实验我不仅掌握了多种时序逻辑电路的设计方法，还培养了系统地分析和调试技能。通过反复调试和观察波形，能够更快地发现和解决问题。实验提升了我编写高质量 Verilog 代码的能力，也为将来设计更复杂的结构奠定了坚实基础。